

Контекст

Необходимо реализовать небольшое приложение для бронирования переговорных комнат внутри компании.

Это диагностическая задача. Её цель — не создать production-ready сервис, а показать, как ты:

- анализируешь требования;
- выделяешь бизнес-правила;
- проектируешь код;
- распределяешь ответственность;
- обрабатываешь ошибочные сценарии;
- тестируешь решение;
- объясняешь принятые решения.

Ограничение по времени

На выполнение задачи нужно потратить **не более 4 часов**.

Если за это время не удалось реализовать все требования, остановись и зафиксируй:

- что реализовано;
- что осталось;
- как бы ты продолжил работу;
- почему выбрал именно такой порядок реализации.

Не нужно тратить дополнительное время на доведение оформления до идеального состояния.

Функциональные требования

1. Создание переговорной комнаты

Система должна позволять добавить переговорную комнату.

Для комнаты необходимо указать:

- уникальный идентификатор;
- название;
- вместимость;
- список доступного оборудования.

Примеры оборудования:

- телевизор;
- проектор;
- доска;
- система видеоконференций.

Правила

название комнаты не должно быть пустым;
вместимость должна быть положительным числом;
нельзя создать две комнаты с одинаковым идентификатором.

2. Создание бронирования

Пользователь должен иметь возможность забронировать переговорную комнату.

Для бронирования необходимо указать:

уникальный идентификатор бронирования;
идентификатор комнаты;
идентификатор организатора;
дату и время начала;
дату и время окончания;
количество участников.

Правила

Бронирование можно создать, только если:

указанная комната существует;
время начала раньше времени окончания;
количество участников положительное;
количество участников не превышает вместимость комнаты;
выбранный временной интервал не пересекается с другим действующим бронированием этой комнаты.

Бронирования разных комнат могут проходить одновременно.

Два бронирования считаются допустимыми, если одно начинается ровно в момент окончания другого.

Например:

```
Бронирование А: 10:00–11:00
Бронирование В: 11:00–12:00
```

Такие бронирования не пересекаются.

Следующие бронирования пересекаются:

```
Бронирование А: 10:00–11:00
Бронирование В: 10:30–11:30
```

3. Отмена бронирования

Система должна позволять отменить существующее бронирование.

Правила

нельзя отменить несуществующее бронирование;

отменённое бронирование не должно блокировать соответствующий временной интервал;
повторная отмена уже отменённого бронирования должна завершаться бизнес-ошибкой;
информация об отменённом бронировании должна сохраняться.

Удалять бронирование полностью не нужно.

4. Получение расписания комнаты

Система должна позволять получить список бронирований конкретной комнаты за указанный период.

На вход передаются:

- идентификатор комнаты;
- начало периода;
- окончание периода.

В результат должны попадать бронирования, временной интервал которых пересекается с запрошенным периодом.

Результат должен быть отсортирован по времени начала бронирования.

В расписании должны присутствовать как действующие, так и отменённые бронирования. Статус бронирования должен быть доступен вызывающему коду.

Формат реализации

Можно использовать основной язык программирования, с которым ты работаешь.

Не требуется

В рамках задачи не нужно реализовывать:

- HTTP API;
- пользовательский интерфейс;
- авторизацию;
- подключение к PostgreSQL или другой базе данных;
- Docker;
- Kafka;
- микросервисную архитектуру;
- логирование и метрики;
- интеграцию с внешними календарями.

Данные можно хранить в памяти процесса. Или как тебе удобно.

Допускается использование стандартной библиотеки языка и обычного тестового фреймворка.

Требования к коду

Решение должно:

- запускаться локально;

- не требовать ручного редактирования исходного кода для проверки основных сценариев;
- явно обрабатывать нарушения бизнес-правил;
- различать успешный результат и ошибочную ситуацию;
- сохранять состояние комнат и бронирований хотя бы в памяти процесса.

Конкретную архитектуру, набор классов, интерфейсов и паттернов необходимо выбрать самостоятельно.

Не нужно добавлять абстракции только ради демонстрации паттернов проектирования.

Что необходимо сдать

1. Исходный код

Репозиторий или архив с реализацией.

2. README

В `README.md` нужно указать:

Как запустить решение

- требования к окружению;
- команда установки зависимостей;
- команда запуска тестов.

Какие бизнес-правила реализованы

Кратко перечислить основные правила системы.

Какие решения были приняты

Ответить на вопросы:

- Какие основные части или объекты ты выделил в системе?
- Где находятся правила создания и отмены бронирования?
- Кто отвечает за проверку пересечения временных интервалов?
- Как представлено состояние бронирования?
- Как вызывающий код узнаёт о нарушении бизнес-правила?
- Какие зависимости есть между основными частями решения?
- Что пришлось бы изменить, чтобы хранить данные в PostgreSQL?
- Какие недостатки или спорные места ты видишь в своей реализации?
- Что бы ты улучшил, имея ещё четыре часа?

Объём README не должен превышать примерно двух страниц.

3. Краткий отчёт о работе

В конце README указать:

Фактически потрачено времени:

Что успел реализовать:

Что не успел:

Самая сложная часть:

Решение, в котором больше всего сомневаюсь:

Правила использования AI и внешних материалов

Разрешено пользоваться документацией языка и библиотек. Не следует использовать AI-ассистентов для проектирования или генерации реализации. После задания мы вместе разберём решение и при необходимости используем AI уже как рабочий инструмент.